dbx-flame
v0.2.2

Onboard a new dataset with a **configuration file, not code**, and trace every record in Silver and Gold back to the **exact file or table** it came from.

01

Configuration-Driven Ingestion

Each dataset is a short YAML block in a workflow: its source, how to write it, and where. One engine reads CSV and JSON files, Delta tables and federated databases (SQL Server, Postgres and more, through Unity Catalog). Five write patterns cover every layer: append, full refresh, upsert, history (SCD2) and complete replay. A table config sets each column's type, comment and tags. Every combination is checked before any data is read.

- New dataset = one YAML file
- Invalid setups are rejected before they run
- The same patterns serve every layer

02

Built-in Data Quality

Each batch passes a quality gate before it is written. Your rules live in a Databricks DQX ruleset, versioned with the pipeline: errors stop the batch, warnings are logged. Framework checks are always on, for files without an export stamp, values a type conversion would silently turn into NULL, unparseable dates and unexpected new columns.

- Bad batches never land
- Silent data loss is caught at the source
- Rules versioned with the pipeline

03

Record-Level Traceability (ALCOA+)

Every row carries where it came from and when it was exported. That record is written once, when the data enters the platform, and never overwritten as the row moves to Silver and Gold. Table lineage shows which tables feed which; this shows the exact file or table read behind each row. Every run writes an audit record with its row counts, and exports carry a checksum per file and a manifest.

- Any Gold row traced to its source
- Audit-ready by design
- Every run logged, success or failure

04

Complete History, Correctly Ordered

History-tracking tables keep every version of a record with its validity window. A backlog of exports is replayed one export at a time, in order, so no intermediate state is lost. Deletes are recorded and flagged, never physically removed, and full snapshots retire records the source has dropped. Re-running a job never changes the result.

- Every change preserved
- Safe re-runs, no duplicates
- No physical deletes, ever

05

Deployed and Tested as Code

dbx-flame ships as a Python package and deploys through Databricks Declarative Automation Bundles. Environments are separated by catalog naming, so moving from dev to prod is a deployment, not a rebuild. Every change runs a unit suite and a platform suite of real Databricks jobs that generate their own data and check the resulting tables.

- dev → prod in one command
- Tested on real Databricks, every change
- No notebooks, no hidden logic

Next on the roadmap: SAS ingestion · sample Gold views over Silver · JSON exports · read consistency for federated sources

Databricks · Delta Lake · DQX · Unity Catalog